Twitter Clone — Code Defense Questions

Oral examination / project handover

Advanced Programming — Dr. Saeed Reza Kheradpisheh

The following 30 questions are designed to verify that the team understands every layer of its own implementation: the socket layer and protocol, server concurrency, the JDBC/DAO layer and SQL, object-oriented design, and the JavaFX client. Each student should be able to justify the specific design choices in the code, not merely describe what it does.

1. Architecture & Networking

Q1. On the server, each connection is handled by a `ClientHandler` that `implements Runnable` and is submitted to a fixed thread pool created with `Executors.newFixedThreadPool(20)`. Why a bounded pool rather than creating a new `Thread` per client, and what happens when the pool is exhausted?

Q2. The protocol reads and writes one JSON object *per line* using `BufferedReader.readLine()` and `PrintWriter.println()`. What assumption does this make about the message content, and how would the protocol break if a tweet's text contained a newline character? How is that avoided?

Q3. Every request carries a `requestId` (a UUID) that the server echoes back on the response. What concrete problem does this solve on the client compared to matching responses by their packet `type`?

Q4. In `ClientHandler.handleLine`, the call to `Dispatcher.dispatch` is wrapped in a `try/catch (RuntimeException)`. Why catch a `RuntimeException` here specifically, and what would happen to the client's connection if this catch were removed and a handler threw an unchecked exception?

Q5. Real-time delivery is implemented by `ConnectionRegistry`, a class with only `static` members and a private constructor. Walk through what happens, step by step, from the moment user A posts a tweet to the moment it appears in user B's feed. Which users receive the `NEW_TWEET_PUSH`?

Q6. A push message (e.g. `NEW_TWEET_PUSH`) has no `requestId`, while a response does. On the client, how does `NetworkService` use this difference to decide whether to invoke a pending callback or a push listener?

2. Concurrency & Thread Safety

Q7. `Authenticator` and `ConnectionRegistry` both store their state in a `ConcurrentHashMap`. Which threads access these maps simultaneously, and what specific bug could occur if a plain `HashMap` were used instead?

Q8. `ClientHandler.sendPacket` is declared `synchronized`. Given that each client has its own handler, which two events could otherwise write to the same socket concurrently, and what would the symptom of that race be on the wire?

Q9. On the client, the background listener thread never updates the UI directly — it wraps every callback in `Platform.runLater(...)`. What rule of JavaFX does this respect, and what exception would you expect if the UI were updated from the listener thread instead?

Q10. The database is accessed through a HikariCP connection *pool* exposed as a singleton `DatabaseConnection`. Why is a pool preferable to opening a fresh `DriverManager.getConnection()` in each DAO

method, and how does `try-with-resources` interact with the pool (does `close()` really close the socket to Postgres)?

3. Database, DAO & SQL

Q11. Every DAO builds its queries with `PreparedStatement` and parameter binding (`setInt`, `setString`) rather than string concatenation. Show, with a concrete example from `SearchDAO`, how this prevents SQL injection in the `ILIKE` search.

Q12. `TweetDAO.createTweet` sets `conn.setAutoCommit(false)`, performs three inserts (tweet, media, hashtags), then commits — and rolls back on any `SQLException`. Why must these three inserts be one transaction, and what inconsistent state could arise without it?

Q13. The feed query flattens retweets using `JOIN tweets d ON d.id = COALESCE(r.retweet_of, r.id)`. Explain what `r` and `d` represent, and why `COALESCE` produces the correct "display tweet" for both an original tweet and a repost.

Q14. The tweet-reading queries compute `liked` and `retweeted` per row with correlated `EXISTS` subqueries parameterized by the viewer's id. Why compute these on the server in one query instead of returning raw tweets and letting the client ask separately?

Q15. `FollowDAO.follow` and `LikeDAO.like` use `INSERT ... ON CONFLICT DO NOTHING`. What property does this give the operation, and why does it matter if the same "Follow" button is clicked twice quickly by the same user?

Q16. The `follows` and `likes` tables use a *composite* primary key `(follower_id, following_id)` / `(user_id, tweet_id)` instead of a surrogate `SERIAL id`. Justify this choice.

Q17. `TweetDAO.deleteTweet` uses a `WITH RECURSIVE` CTE to collect a tweet and its entire reply subtree, then deletes their likes and the tweets themselves. Why is the recursion necessary, and why can the parent and its child rows be removed in a single `DELETE ... WHERE id = ANY(?)` without a foreign-key violation?

Q18. The schema was extended *additively* (using `ADD COLUMN IF NOT EXISTS` and `CREATE TABLE IF NOT EXISTS`) and is re-run on every server startup by `SchemaInitializer`. What does "idempotent" mean here, and why is re-running the whole script on each boot safe?

Q19. `MediaDAO.getForTweets` and `HashtagDAO.getForTweets` fetch data for many tweets in one query using `WHERE tweet_id = ANY(?)` with a `java.sql.Array`. What performance problem (hint: "N+1") does this avoid compared to querying inside the row-mapping loop?

Q20. Passwords are stored via `PasswordUtil.hash` which calls `BCrypt.hashpw(plain, BCrypt.gensalt())`. Where is the salt stored, why is BCrypt preferable to a plain SHA-256 hash, and why can two users with the same password have different `password_hash` values?

4. Protocol & Serialization

Q21. The `Packet.payload` field is typed as `JsonElement`, but the getter returns a `JsonObject`. This was changed from a plain `JsonObject` field. What runtime error did the original type cause when a message arrived with `"payload": null`, and how does the current design fix it?

Q22. The `User` model marks `passwordHash` as `transient`. What does `transient` do during Gson serialization, and why is this a security-relevant decision when a `User` is sent to the client?

Q23. `Packet` exposes static factory methods `request`, `ok`, `error`, and `push`. What is the benefit of these factories over calling the constructor directly, and which one attaches a `requestId`?

Q24. The tweet character limit (280) lives in `shared/Protocol.MAX_TWEET_LENGTH` and is enforced both in the JavaFX `Composer` and again in `TweetHandler.handleCreateTweet`. Why validate on *both* sides rather than trusting the client?

5. Object-Oriented Design & Patterns

Q25. Name the design pattern behind each of the following and explain why it fits: (a) `DatabaseConnection/Net` (b) `UserDAO/TweetDAO/...`; (c) `Dispatcher`; (d) `ConnectionRegistry` together with the client's push listeners.

Q26. The `Dispatcher` centralizes authentication: for every packet type except `PING/REGISTER/LOGIN` it calls `Authenticator.validate(token)` and passes the resolved `userId` to the handler. What are the advantages of doing this in one place versus checking the token inside each handler?

Q27. `TweetDAO` defines a small functional interface `Binder` so that a single private `query()` method can serve every read (feed, profile, replies, search) by letting each caller bind its own parameters. Which OOP principle does this illustrate, and what would the code look like without it?

6. JavaFX Client

Q28. A `TweetCard` performs an "optimistic" like: it increments the count in the UI immediately, then sends `LIKE_TWEET` and *reverts* if the server responds with an error. What is the UX benefit, and how does the card reconcile its state when a `LIKE_PUSH` later arrives from another user's action?

Q29. Dark mode is implemented by adding/removing a `"dark"` style class on the scene root (`Theme.refresh`), and the stylesheet defines CSS variables like `-fx-bg` under `.root` and `.root.dark`. Why is this approach preferable to setting inline styles on individual controls when the theme toggles?

Q30. Image attachments are read from disk, Base64-encoded into a `data:` URI in the `Composer`, sent inside the JSON payload, stored as `TEXT` in the `media` table, and decoded back with `UiUtils.decodeImage`. Discuss the trade-offs of embedding images as Base64 in the database versus storing files on disk and keeping only a path/URL. What is the effect on payload size, and why does the code cap the encoded length?

Tip for graders: ask the student to open the referenced file and point to the exact lines while answering. Strong answers cite the reason *for a choice (safety, concurrency, performance, UX), not just the mechanics.*